CONDITIONAL STATEMENT
AND
CONTROL STATEMENT
In Python

By Swarnima Mitra

Sequential Control Flow

Till now, we are using sequence set of code.

Now, in some cases, we want to either execute only a selected set of statements (i.e., selection control) or execute a set of statements repeatedly (i.e., iterative control). In this case DECISION CONTROL SYSTEM comes into picture.

Decision Control Statement

- Determines the flow of control in a program
- Flow of control – which instruction would be executed next.
- A **Decision Control Statement** is used in programming to control the flow of execution based on conditions. These statements help the program make decisions and execute different blocks of code accordingly.

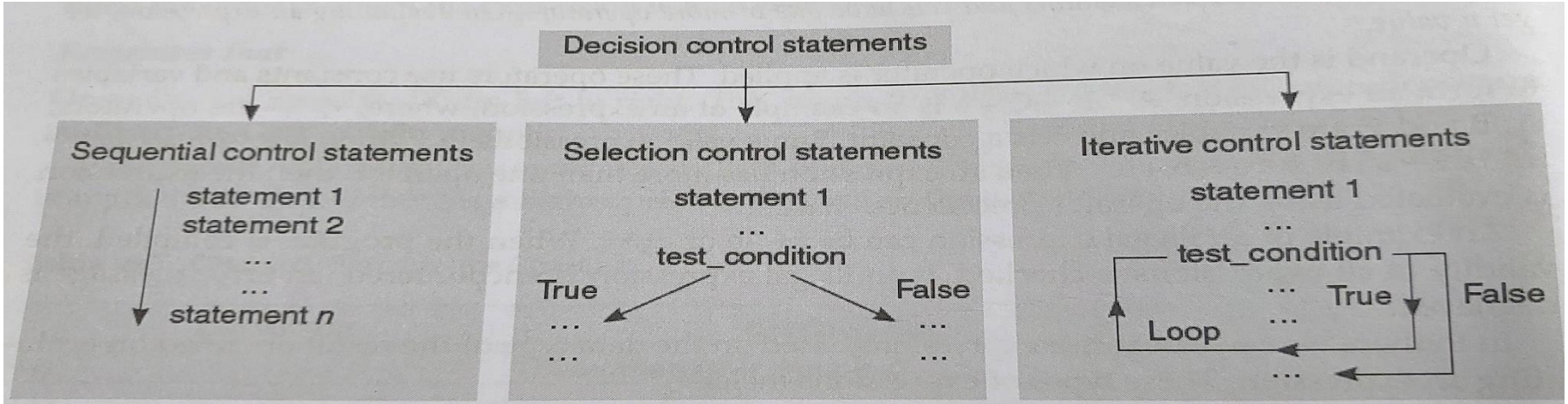


Figure 1.4 Categorisation of decision control statements

Selection/Conditional Branching Statement
(if, if-else, nested, if-elif-else)

- The Program that executes statements selectively based on certain decisions
- Selection Control statements is also called conditional branching statements

NOTE : Python is a case sensitive. So, ‘if’ is not same as ‘IF’ or ‘If’

NOTE : Python uses indentation to form a block of code, other languages like C, C++, use curly braces {} to accomplish the code

CONDITIONAL STATEMENTS (Decision Making)

- The basic decision statements in computer is selection structure.
- The decision is described to computer as a conditional statement that can be answered True or False.

Python language provide the following conditional (Decision making) statements.

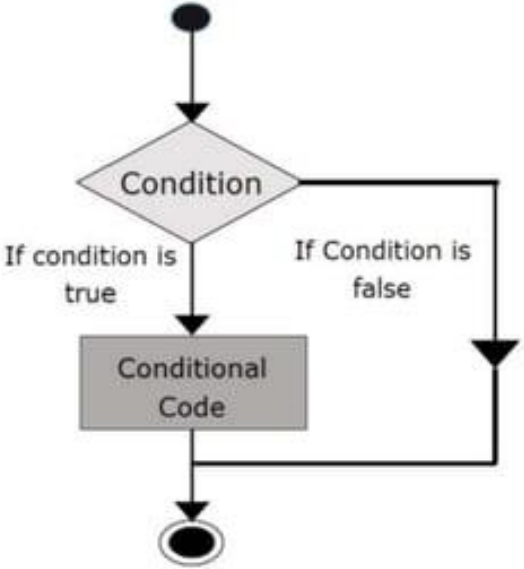
- **if statement**
- **if...else statement**
- **if...elif...else staement**
- **Nested if..else statement**

The if statement

- The **if** statement is a decision making statement. It is used to control the flow of execution of the statements and also used to test logically whether the condition is true or false.

Syntax

```
if test expression:
    statement(s)
```



Example program

```
i=int(input("Enter the number:"))
If (i<=10):
   print(" condition is true")
```

OUTPUT

Enter the number: 9
Condition is true

Program to increment a number if it is positive

```
X = 10   # initialize the value of x

if(x>0):     # test the value of x
             x=x+1  # increment the value of x if it is > 0
print(x)    # Print the value of x
```

OUTPUT:

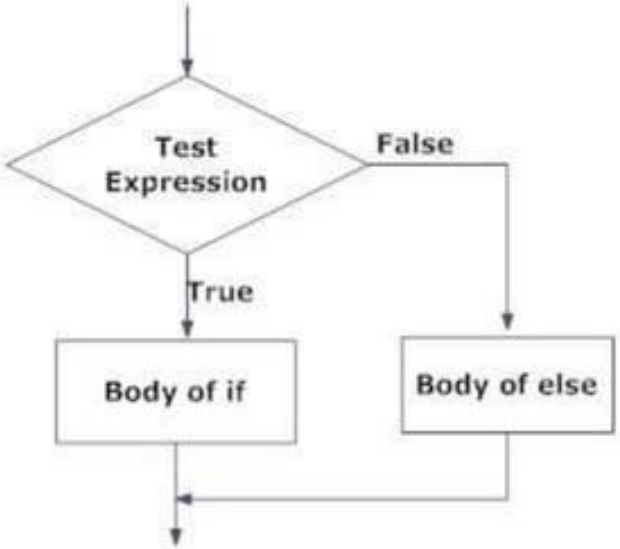
X = 11

If … else statement

- The if…else statement is called alternative execution, in which there are two possibilities and the condition determines wich one gets executed.

Syntax

```
if test expression:
        Body of if
else:
        Body of else
```



Write a program to check if a number is Odd or Even

```
num = int(input("Enter the number:"))
if (num % 2)== 0:
    print ("Given number is Even")
else:
    print(" Given number is Odd")
```

OUTPUT

Enter the number: 9
Given number is Odd

Program to check if a no. is odd or even

```
num = int(input(“Enter a number: ”))
if (num%2 == 0):
        print(“Given number is even”)
else:
        print(“Given number is odd”)
print(“Program executed”)
```

OUTPUT:

Enter a number: 9

Given Number is odd

elif Statements

- elif – is a keyword used in Python in replacement of else if to place another condition in the program. This is called chained conditional.
- Chained conditions allows than two possibilities and need more than two branches.

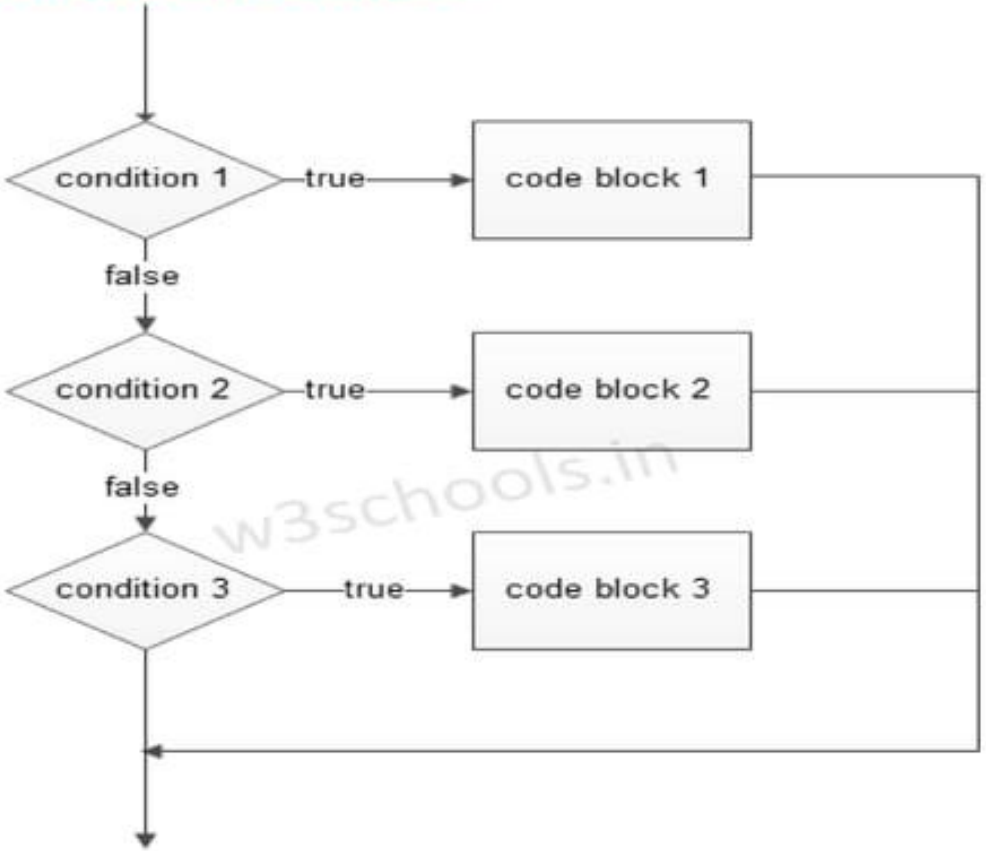
SYNTAX

A series of if and elif statements have a final else block, which is executed if none of the if or elif expression is True.

```
if  expression:
    Body of if
elif  expression:
    Body of elif
else:
    Body of else
```

If-elif-else construct is also known as nested-if construct.

Figure – elif condition Flowchart



w3schools.in

Example: largest among three numbers

```
a = int(input("Enter 1st number:"))
b= int(input("Enter 2nd number:"))
c= int(input("Enter 3rd number:"))
if (a > b) and (a > c):
        print("a is greater")
elif (b < a) and (b < c):
        print("b is greater")
else:
        print("c is greater")
```

OUTPUT

```
Enter 1st number:10
Enter 2nd number:25
Enter 3rd number:15
B is greater
```

Program to check whether a no. entered by the user is negative, positive or 0.

```
num = int(input("Enter a number: "))
if (num == 0):
        print("The value is equal to 0")
elif(num>0):
        print("The number is positive")
else:
        print("The number is negative")
print("Program executed")
```

OUTPUT:

Enter a number: -10

The number is negative

Nested if … else Statements

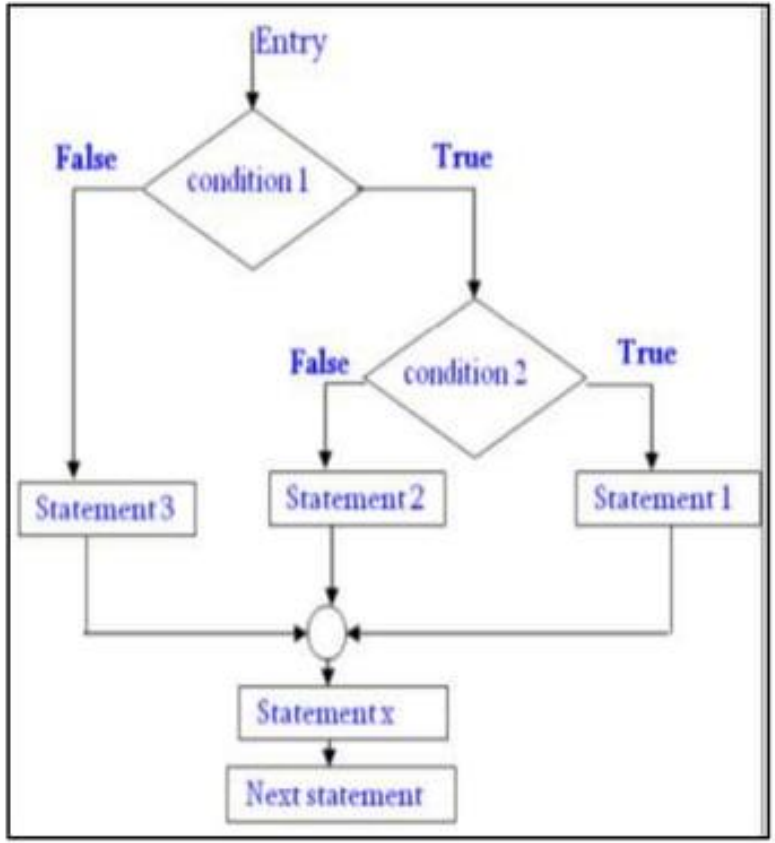
❖ We can write an entire if… else statement in another if… else statement called nesting, and the statement is called nested if.

❖ In a nested **if** construct, you can have an if … elif … else construct inside an if … elif.. Else construct.

A statement that contains other statements is called a **compound statement**.

Syntax

```
if expression1:
        statement(s)
if expression2:
        statement(s)
elif expression3:
        statement(s)
else:
        statement(s)
.
```



- Example program

```
n = int(input("Enter  number:"))
If (n<=15):
   if (n == 10):
        print('play cricket')
   else:
        print('play kabadi')
Else:
   print('Don't play game')
```

OUTPUT

Enter number : 10

Play cricket

Program that prompts the user to enter a number and then print its interval.

```
num = int(input("Enter any number from 0-30"))
if (0 <= num <=30):
        print("It is in the range 0-30")
        if (num >= 0 and num<10):
                print("It is in the range 0-10")
        else:
                print("It is in not in the range of 11-30")
else:
        print("The number is out of range 0-30")
print("Program executed")
```

OUTPUT:

Enter a number: 40

The number is out of range 0-30

Program executed

CONTROL STATEMENT (Looping Statement)

- Program statement are executed sequentially one after another. In some situations, a block of code needs of times.
- These are repetitive program codes, the computers have to perform to complete tasks. The following are the loop structures available in python.

  - **while statement**
  - **for loop statement**
  - **Nested loop staement**

Python supports basic loop structures. Iterative statements are used to repeat the execution of one or more statements.

Iterative statement are used to repeat the execution of a list of statement, depending on the value of an integer.

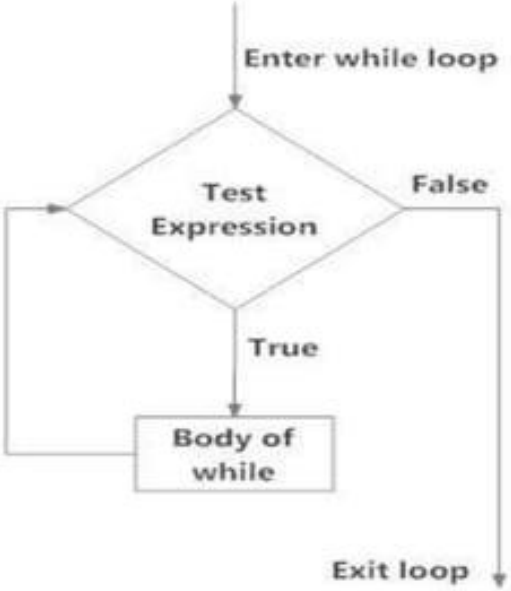
For ex: Printing the First 10 Even Numbers

While loop statement

- A **while** loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

Syntax of while loop

```
while expression:
    statement(s)
```



Using else statement with while loops

- Python supports t have an else statement associated with a loop statement.
- If the else statement is used with a while loop, the else statement is executed when the condition false.

Program to illustrate the else in while loop

```
counter = 0
while counter < 3:
    print("Inside loop")
    counter = counter + 1
else:
    print("Outside loop")
```

OUTPUT
Inside loop
Inside loop
Inside loop
Outside loop

Print the first five numbers

```
i = 1  # initialization of a pointer
while i < 6:    # using loop gives condition
        print(i)   # printing value when condition is true
        i += 1   # Increment of the pointer's value
```

Output»

1
2
3
4
5

Note: remember to increment i, or else the loop will continue forever.

Print a message once the condition is false

```
i = 1
while i < 6:
        print(i)
        i += 1
else:
        print("i is no longer less than 6")
```

Output»

```
1
2
3
4
5
i is no longer less than 6
```

Program to print first 10 numbers using a while loop

```
i = 0
While(i<=10):
        print(i, end = “ ”)
        i = i+1
print(“Program executed”)
```

OUTPUT:

0 1 2 3 4 5 6 7 8 9 10

Program to print sum of first 10 numbers using a while loop

```
num = 0
sum = 0
while(num<=10):
    sum = sum+num
    num = num +1
print(sum)
```

The infinite loop is a loop which never stops running. Its condition is always True

OUTPUT:
55

For loop statement

- The for loop is another repetitive control structure, and is used to execute a set of instructions repeatedly, until the condition becomes false.
- The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. Iterating over a sequence is called traversal.

Syntax

```
for val in sequence:
        Body of for loop
```

For loop flow chart

Addition of number using for loop

```
numbers = [6, 5, 3, 8, 4, 2, 5, 4]
sum1 = 0
for val in numbers:
sum1 = sum1+val
print("The sum is", sum1)
```

OUTPUT

The sum is 37

```
for i in “PYTHON”:
        print(i, end = ‘ ’)
```

OUTPUT>>
P Y T H O N

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

for Loop and for Loop with else

EX-01:

```
genre = ['pop', 'rock', 'jazz']
for i in range(len(genre)):
    print("I like", genre[i])
```

OUTPUT

```
I like pop
I like rock
I like jazz
```

EX-02:

```
genre = ['pop', 'rock', 'jazz']
for i in range(len(genre)):
        print("I like", genre[i])
else:
    print("No items left.")
```

OUTPUT

```
I like pop
I like rock
I like jazz
No items left.
```

RANGE FUNCTION - range()

The range() is a built-in function in Python that is used to iterate over a sequence of numbers. The syntax of range() is:

range(start, end, [step])

Step argument is optional. It can be positive, negative, but not 0.
By default, every number in the range is incremented by 1, but we can specify a different increment using step.
End should be written with +1 index

Example

Using the range() function-

```
for x in range(6):
  print(x)
```

Output>>
0
1
2
3
4
5

Using the start parameter:

```
for x in range(2, 6):
  print(x)
```

Output>>
2
3
4
5

Increment the sequence with 3 (default is 1)

```
for x in range(2, 30, 3):
  print(x, end = ‘ ’)
```

Output>>

```
2
5
8
11
14
17
20
23
26
29
```

Else in For Loop

The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Nested Loops

- Nested loop means that a loop is placed inside another loop. Nested loop is usually get in picture for ‘for loop’.
- The "inner loop" will be executed one time for each iteration of the "outer loop":

Print each adjective for every fruit:

```
adj = ["red",  "tasty"]
fruits = ["apple", "cherry"]

for x in adj:
  for y in fruits:
    print(x, y)
```

OUTPUT>>

```
red apple
red cherry
tasty apple
tasty
cherry
```

Program to print the pattern:

PASS 1- 12345
PASS 2- 12345
PASS 3- 12345
PASS 4- 12345
PASS 5- 12345

```
for i in range(1,6): # Outer loop (i runs from 1 to 5)
        print('PASS', i, "-", end='') # Print "PASS i -" without a newline
        for j in range(1,6):  # Inner loop (j runs from 1 to 5)
                print(j, end=' ') # Print numbers 1 to 5 in the same line
        print()    # Moves to the next line after inner loop finishes
```

Explanation:

- **Outer loop (i)** runs from **1 to 5**, controlling the number of lines.
- On each pass, it prints "PASS i -" (e.g., "PASS 1 -") **without moving to the next line** (end='').
- **Inner loop (j)** runs from **1 to 5**, printing numbers **1 to 5** in the same line.
- After the inner loop completes, print() moves to the next line.
- The process repeats for **each value of i**, creating 5 lines with numbers 1 to 5.

☞ **In short:** This code prints "PASS" followed by the iteration number and numbers 1 to 5 on the same line for each loop cycle.

Homework Question

Write a program to print the following pattern

```
**********
**********
**********
**********
**********
```

```
for i in range(5):
    print()
    for j in range(10):
        print("*", end = ' ')
```

Difference Between for and while Loops in Python

Feature	for Loop	while Loop
Usage	Used when the number of iterations is **known**.	Used when the number of iterations is **unknown** and based on a condition.
Condition	Iterates over a sequence (list, range, string, etc.).	Runs as long as a condition is **True**.
Control	Stops automatically when it reaches the end of the sequence.	Requires manual control (increment/decrement) to avoid infinite loops.
Readability	Easier to read and use when looping over a sequence.	Useful for conditions where termination isn't predefined.

Using a for Loop
(When Iteration Count is Known)

```
for i in range(1, 6):
        print(i, end = ‘ ’)
```

Output:
1 2 3 4 5

✅ **Best for:** Looping a specific number of times.

Using a while Loop
(When Iteration Count is Unknown)

```
i = 1
while i <= 5:
        print(i. end = ‘ ’)
        i += 1     # Manually incrementing
```

Output:
1 2 3 4 5

✅ **Best for:** Running until a certain condition is met.

BREAK STATEMENT

Break statement is used to terminate the execution of the nearest enclosing loop in which it appears.
When the compiler encounters a break statement, the control passes to the statement that follows the loop.
The break keyword is used to break out a for loop, or a while loop.

Eg.,
Break the loop when x is 3, and see what happens with the else block:

```
for x in range(6):
    if x==3:
            break

    print(x)
else:
    print("Finally finished")
```

OUTPUT

```
0
1
2
```

The else block will NOT be executed if the loop is stopped by
a break statement.

```
while...
        .....
        if condition:
                break
        ......
.......
```

Transfers control out of the while loop

```
for...
        .....
        if condition:
                break
        ......
.............
```

Transfers control out of the for loop

```
for...
        ............
        for...
                ........
                if condition
                        break
                ........
                ........
```

Transfers control out of inner for loop

#Program to demonstrate the break statement

```
i = 1
while i <= 10:
        print(i, end = ' ')
        if i == 5:
                break
        i = i + 1
print("\n Done")
```

OUTPUT

1 2 3 4

Done

Python program to show how to control the flow of loops with the break statement

```
for num in range(1, 11):  # Loop from 1 to 10
    print(num)
    if num == 5:  # Condition to break the loop
        print("Breaking the loop at", num)
        break  # Exits the loop

print("Loop terminated")
```

OUTPUT

1 2 3 4 5

Breaking the loop at 5

Loop terminated

CONTINUE STATEMENT

Like Break, continue statement can only appear in the loop.
When compiler encounters this, then rest of statements(in loop) get skipped and unconditionally control transferred to loop – continuation portion.
With the continue statement we can stop the current iteration, and continue with the next

Eg.,

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

OUTPUT

```
1
2
4
5
6
```

Note that number 3 is missing in the result

```
while(...)
    .....
    If condition:
        continue
    ......
.......
```

Transfers control to the condition expression of the while loop

```
for(...)
    .....
    if condition:
        continue
    ......
.................
```

Transfers control to the condition expression of the for loop

```
for(...)
    .............
    for(...)
        ........
        if condition:
            continue
        ........
    ........
```

Transfers control to the condition expression of the inner for loop

Program to demonstrate the break statement

```
for i in range(1, 11):
    if(i==5):
        continue
    print(i, end = '')

print("\n Done")
```

OUTPUT

```
1234678910
Done
```

Differences Between break and continue Statements

Feature	break	continue
Function	Terminates the loop completely.	Skips the current iteration and moves to the next one.
Usage	Used when we want to stop the loop early.	Used when we want to skip specific iterations but continue looping.

Example: Using break and continue

```
for num in range(1, 6):
if num == 3:
        break # Stops the loop when num is 3
        print(num)
print("Loop exited due to break")
```

```
for num in range(1, 6):
        if num == 3:
            continue # Skip iteration of 3
        print(num)
print("Loop completed")
```

PASS STATEMENT

Pass Statement is used when a statement is required syntactically, but no instruction has to be executed.
If specifies a *null operation* or simply **N**o **Op**eration (NOP) statement.
Nothing happens when the pass statement is executed.

Eg.,

```
# Program to demonstrate pass statement
for i in range(5):
    pass # The statement is doing nothing
    print(i, end = " ")
print("\n Done")
```

OUTPUT

```
0 1 2 3 4
Done
```

Nothing will change.

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

```
for x in [0, 1, 2]:
  pass
  print(x)

# having an empty for loop like this, would raise an error without the pass statement
```

What will be the result of the following code:

```
for x in range(3):
  print(x)
```

a. 0 1 2
b. 0 1 2 3
c. 1 2 3

Answer:
a. 0 1 2